

AI IN DATA SCIENCE • CHAPTER 2

Lecture 6

Data Collection & Preprocessing

Part 2 — Preparing Data, Engineering Features & Training Models

Based on Aurélien Géron — Hands-On Machine Learning, 3rd Ed., Chapter 2 • github.com/ageron/handson-ml3

Today's Roadmap

1

Data Cleaning & Categorical Encoding

2

Fixing Skew & Geographic Similarity Features

3

Custom Transformers & Feature Scaling

4

Concise Pipelines (make_pipeline / ColumnTransformer)

5

Select and Train a Model

Picking Up From

Lecture 5's stratified split:

strat_train_set

strat_test_set

We work only on the training predictors.

Learning Objectives

- ✓ Clean missing values with SimpleImputer
- ✓ Encode categorical attributes — OrdinalEncoder vs. OneHotEncoder
- ✓ Fix skewed distributions with a log transform
- ✓ Engineer a geographic similarity feature using k-Means + an RBF kernel
- ✓ Write your own custom Scikit-Learn transformer
- ✓ Chain everything into one concise Pipeline / ColumnTransformer
- ✓ Train and honestly compare several candidate models

First: Separate Predictors from Labels

```
housing = strat_train_set.drop("median_house_value", axis=1)
housing_labels = strat_train_set["median_house_value"].copy()
```

Why now, before any cleaning or scaling?

- Some preparation steps (e.g. an imputer's median) must be learned only from predictors
- If the label leaks into feature preparation, you get target leakage — an unrealistically optimistic model
- This mirrors production: at prediction time you never have the label at all

Data Cleaning — Handling Missing Values

Recall: `total_bedrooms` had missing values in the training set. Most ML algorithms cannot handle NaNs.



Drop rows

```
housing.dropna(subset=["total_bedrooms"])
```

Loses data points entirely



Drop attribute

```
housing.drop("total_bedrooms", axis=1)
```

Loses a potentially useful feature



Impute (fill in)

```
median = housing["total_bedrooms"].median()
```

```
housing["total_bedrooms"].fillna(median,  
                                inplace=True)
```

Keeps the most information — our choice

SimpleImputer & the Fit/Transform Golden Rule

```
from sklearn.impute import SimpleImputer

imputer = SimpleImputer(strategy="median")
housing_num = housing.drop(
    "ocean_proximity", axis=1)
imputer.fit(housing_num)
X = imputer.transform(housing_num)
```

- `fit()` learns the median of every numeric column from the TRAINING data only
- `transform()` applies those exact medians — reusable later on the test set or production
- Never call `.fit()` on the test set — that would leak test-set statistics into your model

Golden rule, repeated all course long: fit on train, transform everywhere.

Handling Text & Categorical Attributes

```
ocean_proximity.value_counts()
```

<1H OCEAN	7,276
INLAND	5,263
NEAR OCEAN	2,124
NEAR BAY	1,847
ISLAND	5

OrdinalEncoder — be careful!

- Maps each category to an integer: 0, 1, 2, 3, 4
- Silently implies an ORDER that doesn't exist ("INLAND" isn't between two other categories)
- A model may learn that numerically close codes are "similar" — meaningless here

→ Use *OneHotEncoder* instead for unordered categories like this one (next slide).

OneHotEncoder — the Safe Default

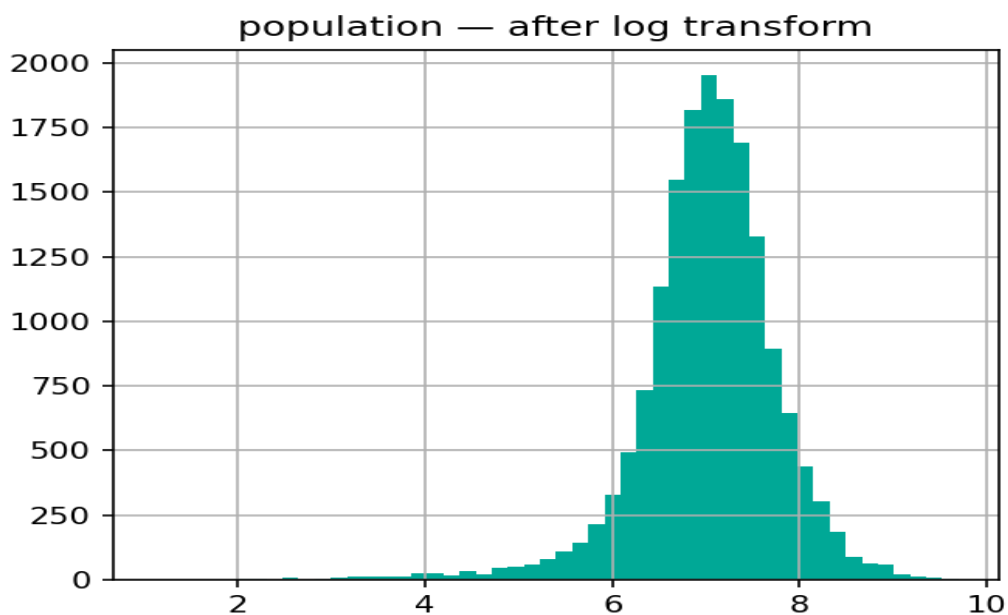
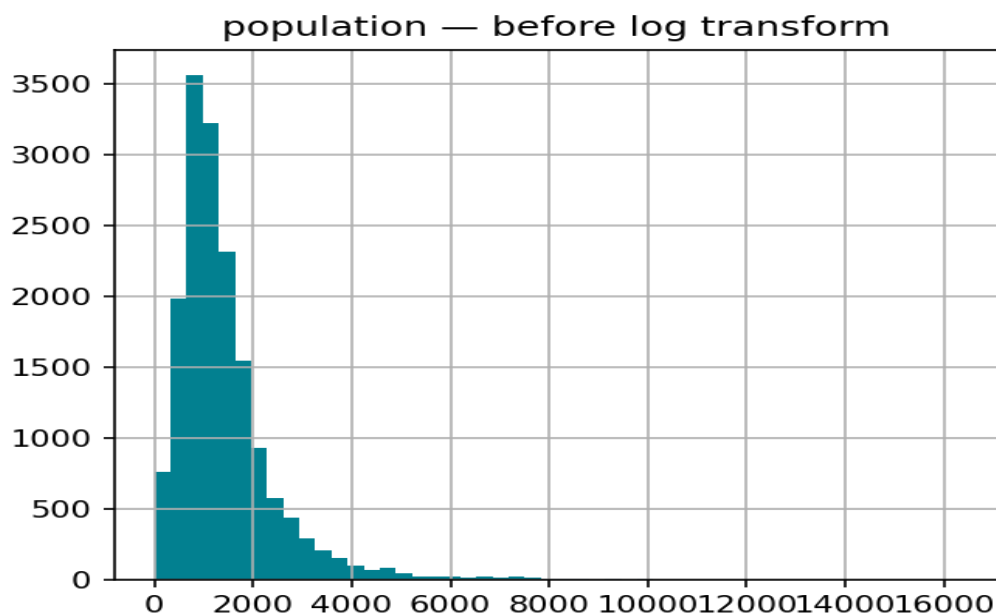
One binary column per category. No false ordering implied.

ocean_proximity	<1H OCEAN	INLAND	ISLAND	NEAR BAY	NEAR OCEAN
<1H OCEAN	1	0	0	0	0
INLAND	0	1	0	0	0
NEAR BAY	0	0	0	1	0
NEAR OCEAN	0	0	0	0	1

Trade-off: more columns. Fine here (5 categories) — can be costly with high-cardinality categorical variables. `handle_unknown="ignore"` also protects against unseen categories at prediction time.

Fixing Skewed Distributions with a Log Transform

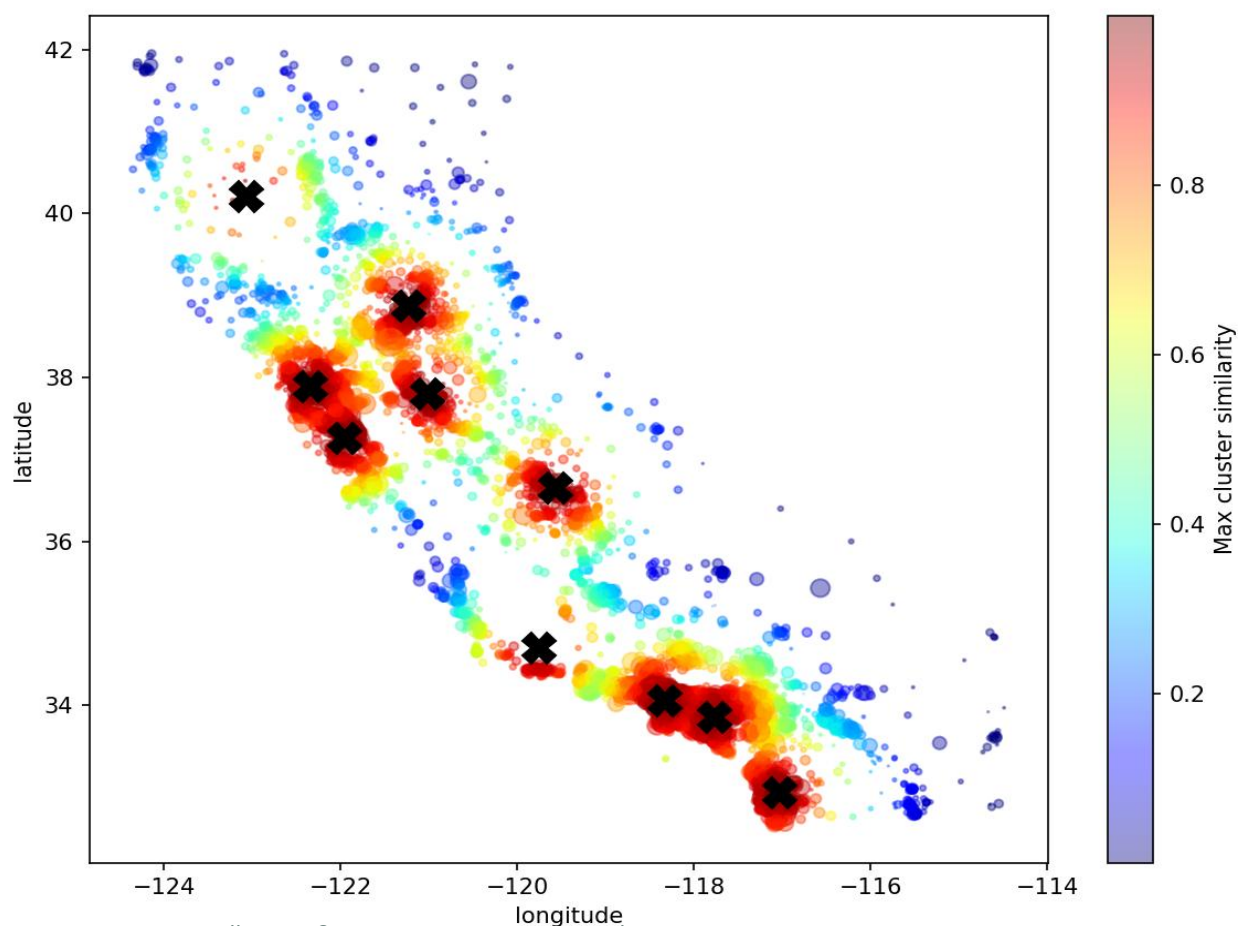
Recall Lecture 5's histograms: total_rooms, total_bedrooms, population, and households are all heavily right-skewed. Many models work best on symmetric, bell-shaped features.



```
FunctionTransformer(np.log, feature_names_out="one-to-one")
```

Geographic Similarity Features (3rd-Edition Technique)

Rather than feeding raw latitude/longitude into a linear model, measure how similar each district is to a handful of geographic “hot spots,” found by k-Means and scored with a Gaussian RBF kernel.



- Black X marks = k-Means cluster centers, weighted toward expensive districts (sample_weight=housing_labels)
- Color = each district's similarity (0–1) to its nearest hot spot
- This blends Lecture 4's unsupervised k-Means into a supervised feature-engineering step

Implementing ClusterSimilarity as a Custom Transformer

```
class ClusterSimilarity(BaseEstimator,
                        TransformerMixin):
    def __init__(self, n_clusters=10,
                  gamma=1.0, random_state=None):
        self.n_clusters = n_clusters
        self.gamma = gamma
        self.random_state = random_state

    def fit(self, X, y=None):
        self.kmeans_ = KMeans(self.n_clusters,
                               n_init=10, random_state=self.random_state)
        self.kmeans_.fit(X, sample_weight=y)
        return self

    def transform(self, X):
        return rbf_kernel(X, self.kmeans_.cluster_centers_,
                           gamma=self.gamma)
```

- `fit()` runs k-Means once, storing the cluster centers
- `transform()` converts distance-to-center into a 0–1 similarity score via an RBF kernel
- `n_clusters`, `gamma` are hyperparameters — tunable with `GridSearchCV` later
- Same recipe as any custom transformer: inherit `BaseEstimator` + `TransformerMixin`

The Same Recipe, Reused: Attribute Combinations

ClusterSimilarity is one example. The same fit()/transform() recipe works for simpler engineered features too — like the ratios from Lecture 5.

```
class CombinedAttributesAdder(
    BaseEstimator, TransformerMixin):

    def __init__(self,
        add_bedrooms_per_room=True):
        self.add_bedrooms_per_room = \
            add_bedrooms_per_room

    def fit(self, X, y=None):
        return self # nothing to learn

    def transform(self, X):
        ... # build & append new columns
        return np.c_[X, new_cols]
```

- fit(): nothing to learn, just return self
- transform(): appends rooms_per_household etc.
- TransformerMixin gives you fit_transform() for free
- BaseEstimator gives automatic hyperparameter handling for GridSearchCV

Feature Scaling

total_rooms can be in the thousands; median_income is roughly 0–15. Most algorithms perform poorly on unscaled features.

Method	Formula	Range	Sensitive to outliers?
Min-max scaling (MinMaxScaler)	$(x - \min) / (\max - \min)$	[0, 1]	Yes — one extreme value compresses everything else
Standardization (StandardScaler)	$(x - \text{mean}) / \text{std}$	unbounded, mean 0, std 1	Much less

Golden rule again: fit the scaler on training data only, then transform both train and test/production data with those same parameters.

The Concise 3rd-Edition Pipeline API

`make_pipeline()` and `make_column_transformer()` auto-name each step, so you skip inventing string names for every single one.

```
log_pipeline = make_pipeline(
    SimpleImputer(strategy="median"),
    FunctionTransformer(np.log, feature_names_out="one-to-one"),
    StandardScaler())

default_num_pipeline = make_pipeline(SimpleImputer(strategy="median"), StandardScaler())
cat_pipeline = make_pipeline(SimpleImputer(strategy="most_frequent"), OneHotEncoder(handle_unknown="ignore"))

preprocessing = make_column_transformer(
    (log_pipeline, ["total_bedrooms", "total_rooms", "population", "households"]),
    (ClusterSimilarity(n_clusters=10, gamma=1.), ["latitude", "longitude"]),
    (cat_pipeline, ["ocean_proximity"]),
    remainder=default_num_pipeline)
```

One Call Does Everything

```
housing_prepared = preprocessing.fit_transform(housing, housing_labels)
```

- Log-transforms skewed counts, computes geographic similarity, one-hot encodes categories, imputes & scales everything else — all in one reproducible call
- `housing_labels` is passed only because `ClusterSimilarity` optionally uses it as a `sample_weight` for k-Means — never as a feature, preserving the golden rule
- Use `fit_transform()` on the training set. Use only `transform()` on the test set and production data.

Select and Train a Model

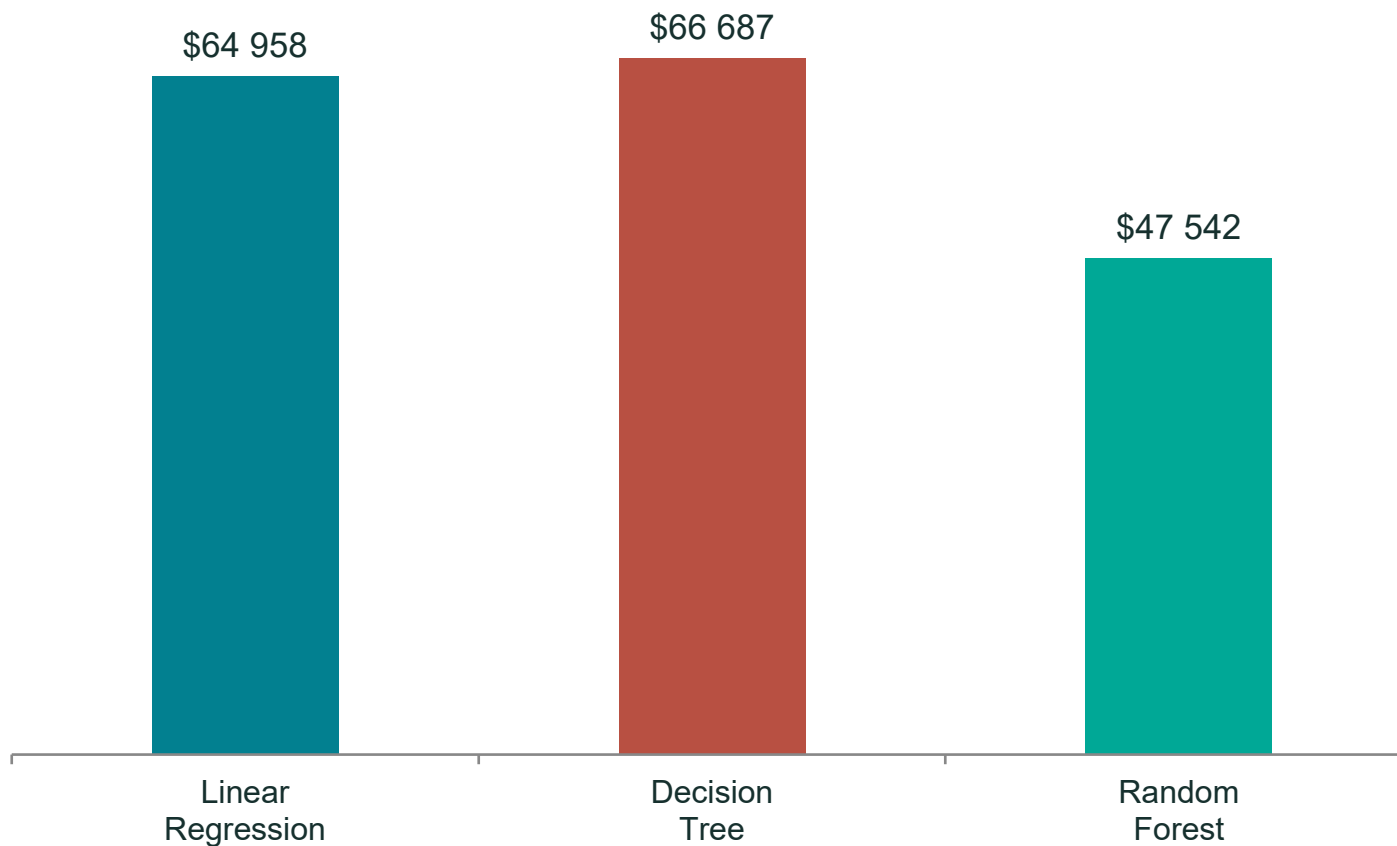
With a clean, fully numeric feature matrix, we can finally train models. Following Lecture 4's No Free Lunch theorem, we try more than one and compare honestly.

Model	Training RMSE	Verdict
Linear Regression	\$64,844	Reasonable, stable
Decision Tree	\$0	Suspicious — near-zero error

A \$0 training error looks amazing — until you recall Lecture 4's overfitting warning. This is almost certainly severe overfitting, not a great model.

We must not trust training-set error. Let's validate honestly with 10-fold cross-validation →

Honest Evaluation — 10-Fold Cross-Validation



- Linear Regression: ~\$64,958
- Decision Tree: ~\$66,687 — worse, despite its \$0 training error!
- Random Forest: ~\$47,542 — an ensemble of many trees, clearly the best of the three

Why Does the Random Forest Win?

A Random Forest trains many Decision Trees on random subsets of the data and features, then averages their predictions.

- Averaging cancels out much of each individual tree's overfitting — the classic bias/variance trade-off in action
- This is a first taste of ensemble learning, a major theme later in the course
- Full hyperparameter tuning (via GridSearchCV / RandomizedSearchCV) is the natural next step, not covered today

Summary

Step	What we did	Why it matters
Data cleaning	SimpleImputer on missing values	Most models can't handle NaNs
Categorical encoding	OneHotEncoder on ocean_proximity	Avoids implying a false order
Skew correction	Log-transform on long-tailed counts	Helps linear models, stabilizes variance
Geographic similarity	k-Means + RBF kernel	Captures spatial price clusters
Custom transformers	BaseEstimator + TransformerMixin	Feature engineering becomes tunable
Pipelines	make_pipeline / make_column_transformer	One reproducible call, train to production
Model selection	Linear vs. Tree vs. Forest, via CV	No Free Lunch: never trust one model or training error

Discussion & What's Next

This closes the Chapter 2: Lecture 6 (clean, engineer, select a model).

For Feedback: <https://forms.gle/BBPPhTzWj5Twx8NF9>